```
     void push(T const& data)
     {
         counted_node_ptr new_node;
         new_node.ptr=new node(data);
         new_node.external_count=1;
         new_node.ptr->next=head.load(std::memory_order_relaxed)
         while(!head.compare_exchange_weak(new_node.ptr-
>next,new_node,

std::memory_order_release,

std::memory_order_relaxed));
     }
     std::shared_ptr<T> pop()
     {
         counted_node_ptr old_head=
            head.load(std::memory_order_relaxed);
        for(;;)
        {
            increase_head_count(old_head);
            node* const ptr=old_head.ptr;
            if(!ptr)
            {
                return std::shared_ptr<T>();
            }
            if(head.compare_exchange_strong(old_head,ptr->next,

std::memory_order_relaxed))
            {
                std::shared_ptr<T> res;
                res.swap(ptr->data);

                int const count_increase=old_head.external_count-2;

                if(ptr->internal_count.fetch_add(count_increase,
                        std::memory_order_release)==-
count_increase)
                {
                    delete ptr;
                }

                return res;
            }
            else if(ptr->internal_count.fetch_add(-1,
                        std::memory_order_relaxed)==1)
            {
                ptr-
>internal_count.load(std::memory_order_acquire);
                delete ptr;
```